System Fundamentals Lab

Activity 10

Performance Optimization, Testing, and Release Verification

Section:
J4T

Submitted by Group IT9:
Masculino, Jhon Darrel C.
Anahaw, Samantha Isabel N.
Policarpio, Lance Angelo A.
Sy, Stephanie Sofia V.

Submitted to:
Ma'am Grachel Eliza C. Greta

November 14, 2026

PART I. PERFORMANCE CONTEXT AND RESOURCE APPLICABILITY

15 Points

Document the environment where your application is being evaluated.

Complete:

Project Title: PAMANA: A Museum Management System Preserving Philippine History and Culture
Current Version:
Target Platform: WebGL / Web Browser
Application Type: Game (2D Educational Adventure Game deployed via WebGL)
Test Device / Computer: Desktop PC
Operating System: Windows 11
Processor, if relevant: Ryzen 7 4750g
Memory / RAM, if relevant: 16gb RAM
Network Connection, if relevant: Broadband Wi-Fi
Profiling / Monitoring Tool Used: Google Chrome DevTools (Performance, Network & Memory tabs), Windows Task Manager, Unity Profiler

Identify the expected behavior of the application.

Performance Area	Expected Behavior	Why It Matters
Startup / Loading	The WebGL build loads from the hosted site and reaches the Main Menu in a few seconds on a school-grade computer, with the progress bar advancing without freezing.	Players open the game directly in a browser, so a slow or stuck first load may make them leave or think the game is broken.
Runtime Responsiveness	Movement, interaction, menus, and quiz battles respond immediately to input, with no noticeable stutter at normal gameplay.	Delayed controls or answer buttons disrupt gameplay and the flow of learning.
Memory Usage	Memory stays stable during play and does not keep growing as the player moves between rooms and battle scenes.	Browsers limit the memory a WebGL game can use, so steady growth could eventually slow or crash the tab.
Save / Load	Progress (character, recovered artifacts, unlocked rooms) saves quickly and loads correctly after login or a page refresh, with no visible freeze.	Players must not lose their progress when they close the tab, refresh, or come back later, and a slow or failed save interrupts gameplay.
Network Response, if applicable	Firebase sign-in, saving, and loading respond within a short time, and the game stays playable (PlayerPrefs fallback) if the connection is slow or lost.	The game depends on an internet connection, and players' connection quality varies.

Then determine which resource areas are relevant to your project.

Resource	Applicable?	Where It Affects the System	Possible Concern
CPU	Yes	Per-frame processing in exploration, animations, quiz battles, and UI, all on the browser's main thread.	Heavy scripts or many animated objects could lower FPS on low-end school computers.
Memory / RAM	Yes	Scene loading and sprite and UI assets held inside the browser tab.	Assets that are not released between scenes could cause memory growth during long play sessions.
Network	Yes	Downloading the build from GitHub Pages, plus Firebase Authentication and Realtime Database requests.	Slow or unstable connections could delay first load, sign-in, and saving.
Disk / Storage	No	The game performs no file reads or writes. PlayerPrefs only stores small save values in browser storage.	None. The game has no disk-intensive operations, and the size of the stored data is negligible.
GPU	No	Rendering is limited to 2D sprites and UI with no heavy shaders or post-processing.	None. A 2D sprite game places minimal load on the GPU.

A **No** or **Not Applicable** response is acceptable when properly justified.

PART II. FUNCTIONAL VALIDATION AND PERFORMANCE BASELINE

20 Points

- Scenario: Main Lobby Exploration, Book Pedestal Proximity Interaction, Field Guide Unlock, and Book Modal UI Access.
- Description: The player loads into the Lobby scene with a selected student explorer avatar, moves across the museum lobby, approaches the newly restored BookPedestal, interacts to unlock the Field Guide, verifies sprite depth sorting around the pedestal, and opens the open-book modal interface via the HUD button and hotkey shortcuts.

A. Functional Validation

Perform at least **three (3) relevant functional tests.**

Requirement / Feature	Test Action / Input	Expected Result	Actual Result	Status
1. Player Character Movement & Collision	Press [W], [A], [S], [D] or Arrow keys to navigate the avatar across the Lobby	Avatar moves smoothly at designated walking speed;	Avatar moved fluidly; wall and pedestal colliders	Pass

Constraints	floor tiles toward the walls and central pedestal.	physical boundaries (RoomWalls and BookPedestal box colliders) prevent the character from walking through solid walls or out of bounds.	stopped character penetration cleanly without clipping.	
2. Book Pedestal Proximity Interaction & Field Guide Unlock	Walk within trigger distance (1.6 units) of the BookPedestal in the Main Lobby and press the interaction key [E] / click the interaction prompt.	Displays interaction banner ("FIELD GUIDE UNLOCKED!"), sets IsBookUnlocked = true in GameProgression. cs, triggers floating book pickup animation, and activates the HUD Book button in the bottom-right corner.	Unlock banner popped up with gold accents; sound played; HUD book button became permanently visible.	Pass
3. Field Guide Book Modal Activation & Page Pagination	Click the HUD Book button (or press [H] / [F1]), click the navigation arrow buttons ([<] Prev / [>] Next), and press [ESC] / close button.	Open-book modal UI overlays smoothly over dimmed background; page turning responds immediately; game timescale pauses cleanly; closing dismisses modal and restores movement.	Book frame opened instantly with zero hitching; arrow buttons registered clicks; closed cleanly returning control to player.	Pass

A component should not be marked **Implemented** only because a script, screen, table, or other technical artifact exists.

Your evidence should reasonably demonstrate that the intended function works.

If an important functional test fails, document the defect before proceeding.

B. Performance Baseline

Record at least **three (3) relevant measurements.**

Possible measurements include:

- Average FPS

- Frame time
- CPU usage
- Memory usage
- Startup time
- Scene loading time
- Save duration
- Load duration
- Database response time
- Network request duration
- Number of network requests
- Amount of transferred data

Complete:

Metric	Test Scenario	Baseline Result	Tool Used
Average Framerate & Frame Time	Continuous player movement around Lobby with active sprite renderers, static props, and UI Canvas.	58.8 FPS (17.01 ms frame time) with occasional dips to 51.2 FPS (19.5 ms) during interaction triggers.	Unity Profiler (CPU Usage) & Chrome DevTools Performance Panel
Per-Frame Sorting Script CPU Overhead	Profiling YSort.LateUpdate() execution across all scene objects (Player, BookPedestal, decorative furniture).	0.82 ms per frame CPU time consumed by dynamic sorting recalculations on stationary objects.	Unity Profiler (CPU Timeline - Deep Profiling mode)
Garbage Collection (GC Alloc) on Modal Toggle	Opening and closing the Field Guide Book Modal and Inventory Bag Modal during exploration.	54.2 KB GC Alloc spike per modal trigger, producing an intermittent 14.8 ms garbage collection pause every 6–8 toggles.	Unity Profiler (GC Alloc counter) & Chrome Memory Allocation Profiler
Cold Startup & WebGL Download Duration	Cold launch of the WebGL build from local HTTP web server with cache disabled.	3.82 seconds initial load time (25.7 MB uncompressed payload: Wasm binary 8.0 MB, Data package 17.6 MB).	Google Chrome DevTools (Network Waterfall, Throttling: Disabled)

Include screenshots or other appropriate measurement evidence.

Whenever possible, keep the following conditions consistent:

- Same device
- Same application version
- Same test scenario

- Same scene or screen
- Similar test duration
- Similar network condition, when applicable

PART III. BOTTLENECK ANALYSIS AND PERFORMANCE DECISION
15 Points

Review your measurements and determine whether a meaningful performance concern exists.

Complete:

Primary Area Reviewed: CPU / Memory
Observed Behavior: During exploration in the Lobby and Hallways, immovable environment props (specifically the BookPedestal, display cases, and static decorative tables) were executing YSort.LateUpdate() on every single rendered frame. Despite remaining at fixed coordinate positions, each static object recalculated sortingOrder = baseOrder - Mathf.RoundToInt((transform.position.y + sortingOffset) * 100f) sixty times per second. Furthermore, modal UI interfaces (such as the newly added Book Modal and Bag Modal) were being created and destroyed dynamically via runtime script instantiations, generating recurring garbage collection allocation spikes (~54 KB) that resulted in micro-stutter frame drops on browser execution.

Supporting Evidence: The Unity Profiler CPU Timeline revealed that YSort.LateUpdate() accounted for ~0.82 ms of main thread execution per frame across all static props in the scene. In the Chrome DevTools Performance timeline, frequent minor GC execution blocks (12–15 ms) coincided with modal opening, momentarily dipping framerates below the 60 FPS target.

Possible Cause:
1. YSort.cs lacked static evaluation logic, forcing every attached GameObject to execute dynamic per-frame coordinate math regardless of whether the object could actually move.
2. UI elements were instantiated on-demand rather than pre-baked inside InGameCanvas.prefab with SetActive(false) toggles.

Effect on User Experience: On dedicated desktop PCs, the impact is subtle, but on low-end school computer lab workstations or student Chromebooks running single-threaded WebGL, unnecessary per-frame CPU load and periodic GC pauses create noticeable hitching and control sluggishness.

Observation	Supporting Evidence	Possible Cause	Priority
Redundant per-frame sorting math on static environmental props	Profiler shows YSort.LateUpdate() executing 60	Missing isStatic guard in YSort.cs to calculate order once	High

	times/sec for immovable objects (BookPedestal, static furniture), taking 0.82 ms CPU time per frame.	on Awake() and disable per-frame updates on non-moving props.	
Garbage collection spikes and frame drops during UI modal instantiation	Unity Profiler recorded 54.2 KB GC Alloc per modal open event; Chrome DevTools showed 14.8 ms GC pause causing frame dip to 51 FPS.	Dynamic runtime instantiation of modal GameObject trees instead of pre-baking modal prefabs inside the persistent Canvas hierarchy.	High
Sprite sorting layering conflict on pedestal table top	Pedestal sprite sorting occasionally rendered in front of resting book item or player sprite when positioned at specific boundary coordinates.	Inconsistent baseline sorting orders and lack of dedicated surface sorting offset component for tabletop props.	Medium

Then select one:

A. Optimization Is Necessary

Your measurements show a meaningful performance concern that should be addressed.

Proceed with a reasonable and controlled improvement.

- While the desktop workstation achieved near-60 FPS, WebGL targets educational school lab environments where single-threaded CPU budgets are strictly limited. Removing redundant per-frame calculations on static props and eliminating GC allocation churn during UI interactions prevents frame stutter and provides a silky, responsive experience across all client devices.

PART IV. PERFORMANCE IMPROVEMENT OR MONITORING PLAN
15 Points

OPTION A. PERFORMANCE IMPROVEMENT

Implement one reasonable optimization addressing the selected concern.

Possible examples:

CPU

- Reduce unnecessary processing every frame.
- Cache appropriate references.

- Reduce repeated calculations.
- Improve an inefficient loop.
- Disable processing when it is not required.

Memory

Reduce unnecessary allocations.
Release unused resources.
Optimize excessively large assets.
Remove unnecessary resource duplication.
Reuse appropriate objects.

Disk / Storage

Reduce repeated file reads.
Reduce unnecessary writes.
Retrieve only required records.
Improve appropriate database queries.
Avoid repeatedly loading unchanged information.

Network

Reduce unnecessary requests.
Retrieve only required data.
Avoid unnecessary synchronization.
Cache suitable information.
Improve request timing.

Complete:

Item	Description
Component Optimized	2D Depth Sorting Architecture (YSort.cs & TableItemSort.cs) and UI Modal System (BookModalUI.cs / InGameCanvas.prefab).
Original Problem	Immovable world props (such as BookPedestal) executed LateUpdate() every frame to recalculate sprite sorting orders, wasting CPU cycles on static entities. UI modals generated dynamic heap allocations and GC spikes upon opening, causing micro-stutters.
Change Implemented	• Static Y-Sort Caching (YSort.cs): Added an explicit [SerializeField] private bool isStatic = false; flag. In Awake(), UpdateSort() calculates the precise sorting order once based on the object's initial world position. In LateUpdate(), an early exit condition if (!isStatic) bypasses execution entirely for stationary props. Moving entities (Player, roaming NPCs) retain isStatic = false.

	• Table Surface Sorting Component (TableItemSort.cs): Created a dedicated component for tabletop items (BookVisual) that locks sorting directly to the parent pedestal's sorting layer + offset (+2), guaranteeing visual consistency without per-frame matrix math • Pre-Baked Inactive UI Hierarchy (InGameCanvas.prefab): Integrated BookModal and BagModal directly into the persistent Canvas prefab with an initial inactive state (SetActive(false)). Toggling now uses zero-allocation canvas activation rather than runtime instantiation.
Expected Improvement	- Reduces YSort CPU execution time by >85% across static props. - Eliminates 100% of runtime GC allocations on modal open/close (0 KB GC Alloc). - Restores a locked 60.0 FPS (16.6 ms frame budget) without GC hitching.
Possible Trade-off	If a developer moves a prop marked isStatic = true via script during runtime, its sorting order will not automatically refresh unless UpdateSort() is called manually (fully acceptable, as museum pedestals and furniture are permanently immovable).

PART V. RE-PROFILING AND FUNCTIONAL REGRESSION TESTING

10 Points

If an optimization was implemented, repeat the same performance test.

A. Performance Comparison

Metric	Before	After	Difference	Interpretation
Average Framerate	58.8 FPS	60.0 FPS	+1.2 FPS (+2.0%)	Achieved locked, perfectly stable 60.0 FPS on WebGL browser display.
Main Thread Frame Time	17.01 ms	15.18 ms	-1.83 ms (-10.8%)	Increased per-frame headroom, keeping well within the 16.66 ms frame budget.
YSort.LateUpdate () CPU Time	0.82 ms / frame	0.08 ms / frame	-0.74 ms (-90.2%)	Static objects bypassed LateUpdate(); CPU time spent on sorting dropped drastically.
Modal Toggle GC	54.2 KB	0.0 KB (Zero	-54.2 KB (-100%)	Pre-baking UI

Alloc Spike		Alloc)		hierarchy into Canvas eliminated all heap allocation churn.
Periodic GC Hitch Duration	14.8 ms pause	0.0 ms (No Pause)	-14.8 ms (-100%)	Eliminated GC-induced frame drops during book and inventory inspection.
Total Runtime Allocated Memory	433.8 MB	432.1 MB	-1.7 MB (-0.4%)	Browser memory heap remained clean, lean, and leak-free.

Do not claim improvement unless the measurements support the conclusion.

If the optimization did not improve the result, explain:

- What happened
- Why the change may not have helped
- What should be investigated next

An unsuccessful optimization may still provide useful technical evidence.

B. Functional Regression Testing

Verify that previously working functionality remains correct after the change.

Feature / Test	Expected Result	Actual Result	Status
1. Static Pedestal Depth Sorting (BookPedestal)	Pedestal renders behind the player when the player stands in front, and in front when the player walks behind it. Book item sits firmly on top of the pedestal.	Layer sorting evaluated accurately on Awake(); player walks smoothly in front and behind; book remains layered on pedestal surface.	Pass
2. Dynamic Player Y-Sorting (Player)	Moving player avatar dynamically updates sortingOrder in real time as the character travels vertically across the room.	Player avatar sorting adjusted continuously in LateUpdate(); no clipping or popping behind floor tiles or walls.	Pass
3. Field Guide Book Modal Toggle & Controls	Book modal opens on HUD click; arrow buttons navigate pages; [ESC] key and close button hide modal and re-enable player movement.	Modal opens instantaneously without lag; arrow buttons paginate cleanly; player movement properly paused and restored upon close.	Pass
4. Game	Unlocking the Field Guide saves	Progress saved and retrieved	Pass

Progress Persistence & Cloud Sync	IsBookUnlocked = true to PlayerPrefs and Firebase Realtime Database; state persists upon scene reload.	successfully; upon scene reload, the pedestal remained empty and the HUD book button remained unlocked.	

Add tests that specifically cover the area affected by the optimization.

For example, if the group changes how saved information is cached, verify that:

- Save still works.
- Load still works.
- Updated information appears correctly.
- Incorrect or outdated information is not displayed.

If no optimization was implemented, repeat one important measurement and functional test to verify that your original baseline is reasonably consistent.

PART VI. NEW BUILD, VERSION, PATCH, AND HOTFIX
15 Points

Determine whether the changes made during development require another application build.

A. WHEN TO CREATE ANOTHER BUILD

A new build should normally be produced when an accepted change affects the application that will be tested or distributed.

- A bug was fixed (e.g., resolving the sprite layering conflict between BookPedestal and BookVisual).
- A performance improvement was implemented (e.g., static caching in YSort.cs and pre-instantiating UI modal prefabs).
- A security correction was made (e.g., input boundary validation on character names).
- A feature was added (e.g., the interactive Field Guide Book modal system).
- Application behavior or configuration changed.
- A new release candidate must be tested.

Do not replace an already distributed application with different content while continuing to identify it as the same build.

B. VERSION NUMBER

Continue using the versioning convention introduced in the previous activity:

MAJOR.MINOR.PATCH

0.1.0 Significant incompatible changes or major milestones

PATCH

Use for corrections that do not introduce a major new feature.

0.3.0 → 0.3.1 Corrections that do not introduce a major new features, such as bug fixes, performance optimizations, and security

MINOR

0.2.0 → 0.3.0 Adding a new working feature or meaningful capability without breaking existing systems

MAJOR

C. NORMAL BUG FIX

Normal Bug Fix: Occurs during regular development or scheduled testing sprints before a designated build candidate is released to production.

Normal Bug Fix Workflow:
Version 0.4.0 → Bug discovered during testing → Bug corrected → Regression test → Version 0.4.1 → New build

D. HOTFIX

Hotfix: A focused, out-of-band correction made when an already released or deployed production version requires an urgent fix to prevent system crashes or data corruption.

Hotfix Workflow: Released Version 1.0.0 → Critical Issue Identified → Hotfix Created → Focused Correction → Test and Regression Test → Version 1.0.1 → New Build → Smoke Test → Corrected Release

Do not classify every normal development bug fix as a hotfix.

E. HOTFIX PROCEDURE

1. Identify the exact affected released version.
2. Start from the corresponding released source code tag.
3. Create a focused hotfix branch or equivalent version-control working area (e.g., hotfix/pedestal-sorting-glitch).
4. Implement only the required correction.
5. Test the corrected feature.
6. Perform regression testing on related functionality.
7. Increment the PATCH version number (0.3.0 → 0.3.1).
8. Produce a new build.
9. Perform a smoke test.
10. Release the corrected build.

11. Integrate the correction back into the continuing development branch.

Example branch:

hotfix/save-game-crash

Avoid adding unrelated new features to an urgent hotfix unless there is a clear technical reason.

F. BUILD DECISION RECORD

Complete:

Item	**Response**
Was application code/configuration changed?	Yes
Is another build required?	Yes
Previous Version	0.1.0
New Version, if applicable	0.2.0
Change Type	Feature / Bug Fix / Performance / Security
Source Reference	66c8ab0
Reason for Version Change	Incremented PATCH version to incorporate core educational features and performance optimizations: (1) integrated all 12 Philippine History topics (24 pages) into the Museum Field Guide Book with green keyword highlights for quiz review; (2) enlarged the Book Modal container (+35.7% scale to 1140×953 px) with 15.5 pt auto-sizing typography for legibility; (3) resolved sprite depth sorting clipping on the Lobby pedestal; (4) eliminated UI garbage collection spikes via pre-instantiated modal hierarchies; and (5) compiled and deployed the verified production WebGL build package to GitHub Pages.

PART VII. BUILD VERIFICATION AND SMOKE TEST
5 Points

If another build was created, perform a basic **smoke test** on the actual packaged application.

A smoke test checks whether the build is stable enough to proceed with further testing or distribution.

At minimum, verify:

Smoke Test	Expected Result	Actual Result	Status
Application launches	Application opens successfully; WebGL loader bar completes; WebAssembly binaries compile without console exceptions.	Application initialized cleanly within 3.8 seconds; WebGL Canvas rendered at native resolution with zero fatal errors.	Pass
Main menu / entry point	User can proceed normally; Main Menu buttons (Start Game, Settings, Credits) and Character Selection screen accept user mouse clicks and navigate to the Lobby.	Character selection loaded; choosing avatar and clicking "CONFIRM & PLAY" smoothly transitioned into the Lobby scene.	Pass
Critical feature	Important function works; avatar walks across Lobby floor, approaches BookPedestal, triggers unlock notification, and toggles the Field Guide Book modal.	Collision detection functioned properly; unlock banner displayed; Book modal opened and paginated with locked 60 FPS.	Pass
Save/load or integration, if applicable	Works correctly; player account authenticates with Firebase; unlocked book flag and progress persist across room doors and page reload.	Cloud REST payload synchronized player progression state; reloading the browser retained unlocked book state.	Pass
Application closes correctly	Exits normally; navigating back to the Main Menu or closing/refreshing the browser tab terminates the WebGL runtime cleanly without memory leaks or hanging processes.	Returning to Main Menu released exploration assets cleanly; tab closed without browser hang or memory retention.	Pass

The smoke test should be performed on the **actual build**, not only through the Unity Editor or development environment.

No new build was produced because no application change requiring another build was made during this activity.

PART VIII. PERFORMANCE AND RELEASE MONITORING

5 Points

Create a short checklist your group can reuse during succeeding builds.:

Check	Expected Condition
Important functional tests	All critical features work as expected.
Critical known defects	Resolved or properly documented.
Performance	No unexplained significant performance regression.

Frame rate	Maintains a stable and acceptable frame rate during gameplay.
CPU usage	No unnecessary processing or unusually high CPU usage.
Memory usage	No abnormal or continuous memory growth.
Garbage collection	No excessive allocations or noticeable GC-related stuttering.
Security-related functions	Input validation, access restrictions, and data handling remain functional.
Version number	Updated when required.
Source reference	Relevant commits and changesets are recorded.
Packaged build	Successfully smoke tested on the intended devices and platforms.
Known issues	Remaining issues and limitations are documented.

Primary Performance Metric to Monitor: Frame Time (milliseconds per frame)

Reason: Frame time indicates how long the game takes to process and render each frame. Monitoring it helps identify performance issues such as frame drops, excessive processing, and interruptions during gameplay. Maintaining consistent frame times contributes to smoother gameplay and helps ensure that the game performs reliably across different devices.

When It Should Be Checked:

- **Major Feature:** After implementing or modifying significant features.
- **Testing Cycle:** During performance testing and optimization.
- **Before Release:** To verify that the packaged build meets performance expectations.
- **After Release:** To identify and address performance issues reported during actual use.

PART IX. INDIVIDUAL CONTRIBUTION RECORD

5 Points

Complete this section individually.

Student Name:	**Policarpio, Lance Angelo A.**
Group Number:	Group IT9
Area Worked On:	**Performance Profiling & Optimization, UI Memory Churn Elimination, 2D Depth Sorting Architecture, WebGL Build Packaging, and Mobile Touch Control Stabilization (Lead Developer)**
Component / File / Asset:	• Assets/Scripts/YSort.cs & TableItemSort.cs (Static 2D depth sorting and table surface locking) • Assets/Scripts/ArtifactInspectionUI.cs & InGameCanvas.prefab (Zero-allocation pre-baked UI modal architecture) • Assets/Scripts/NotificationBannerUI.cs (Accelerated banner transitions: 0.08s slide, 0.06s crossfade, 0.15s dynamic interrupt) • Assets/Scripts/MobileControlsManager.cs, MobileInteractButton.cs, & VirtualJoystick.cs (Touch controls restoration) • Assets/Scripts/DoorInteractor.cs (Interaction re-entrancy frame cooldown) • WebGLTemplates/PAMANA & docs/ (Unity 6 WebGL player build and GitHub Pages deployment)

Specific Contribution:	As Lead Developer for Group IT9, I directed the profiling, optimization, memory stabilization, and release verification pipeline: • **Static Y-Sort Depth Optimization:** Diagnosed per-frame CPU waste in YSort.cs via Unity Profiler Timeline. Implemented an isStatic caching guard that calculates sorting order once on Awake() and bypasses per-frame coordinate math in LateUpdate() for stationary museum pedestals, cases, and furniture. Reduced sorting CPU time by **90.2%** (from 0.82 ms to 0.08 ms/frame). • **Zero-Allocation UI Modal Architecture:** Eliminated recurring 54.2 KB GC allocation spikes and 14.8 ms periodic hitching by converting dynamic modal instantiation into pre-baked inactive hierarchies in InGameCanvas.prefab with SetActive() toggles. • **Notification Acceleration:** Refactored NotificationBannerUI.cs and ArtifactInspectionUI.cs for high-tempo interaction feedback, preventing delay backlogs during rapid pedestal restorations. • **Mobile Touch Control Restoration:** Resolved a critical defect where mobile controls disappeared after artifact inspection. Implemented atomic frame guards, auto-reset in VirtualJoystick.OnDisable(), dedicated mobile Leave choice buttons, and guaranteed HUD re-activation while keeping PC controls 100% untouched. • **Production WebGL Compilation:** Compiled and deployed the verified production WebGL release to GitHub Pages.
Testing Evidence:	• **Unity Profiler (Deep Profiling):** Confirmed locked 60.0 FPS, 15.18 ms frame time, 0.08 ms sorting overhead, and zero heap allocation churn (0 KB GC Alloc) on modal open/close. • **Chrome DevTools (Performance & Memory):** Verified 0 ms GC pauses during exploration and stable 432.1 MB browser heap without memory leaks. • **WebGL Smoke Testing:** Verified launch in 3.82s, responsive controls on PC and mobile touch, and persistent cloud save state.
Commit / Changeset Reference, if applicable:	• 01d8c77: Fix mobile controls disappearing after artifact inspection; add mobile exit options while keeping PC untouched • f2263d7: Make notifications super-fast: 0.08s slide, 0.06s crossfade, dynamic 0.15s interrupt, pedestal restore banners, 0.45s fx • 4224fef: Fix main menu button hover alignment and restore official pixel-art assets • d4e924c: Release v0.1.0: First Controlled Release & WebGL Deployment • Repository URL: https://github.com/nouzen214/PAMANA.git (Branch: main)

Build Responsibility, if applicable:

Lead Build & Deployment Engineer: Solely responsible for configuring Unity 6000.0.54f1 WebGL Player Settings, custom responsive WebGL canvas template, multi-threaded IL2CPP ahead-of-time WebAssembly linking, Brotli compression, generating the production distribution bundle (docs/ and Pamana-v0.1.0-WebGL.zip), and managing continuous deployment to GitHub Pages (https://nouzen214.github.io/PAMANA/).

Briefly answer:

What did you learn about the relationship between functional testing, performance measurement, optimization, and succeeding software builds?

Limit your response to approximately 100 words.

I learned that software engineering is a disciplined cycle where functionality and performance must validate each other. Functional testing ensures features work according to specifications, but it is blind to hidden runtime bottlenecks such as per-frame CPU waste and garbage collection pauses. Performance measurement with profilers provides objective data to identify these bottlenecks, allowing targeted optimizations like static caching and pre-baked UI without guesswork. Finally, succeeding builds require regression and smoke testing to prove that optimizations boosted efficiency without breaking existing gameplay. This rigorous cycle ensures each new build candidate is measurably faster, functionally intact, and genuinely release-ready.

Word Count: 98 words

Performance Optimization & Verification Summary	Observed Outcome
Static Y-Sort CPU Optimization	**PASS (-90.2% CPU overhead; 0.82ms -> 0.08ms)**
Zero-Allocation UI Modal Churn	**PASS (-100% GC Alloc spike; 54.2KB -> 0 KB)**
Periodic Garbage Collection Hitch	**PASS (-100% GC pause; 14.8ms -> 0 ms stutter)**
Framerate & Main Thread Frame Time	**PASS (Locked 60.0 FPS; 17.01ms -> 15.18ms)**
Mobile Controls & Re-entrancy Guard	**PASS (Virtual Joystick & E Button persistent)**
Production WebGL Build & Deployment	**PASS (Build Finished: Success; live on Pages)**

Performance Optimization Evidence (Profiling & Architecture):

The following screenshots and technical logs document the implemented performance improvements, comparative benchmarks, and zero-allocation memory stabilization:

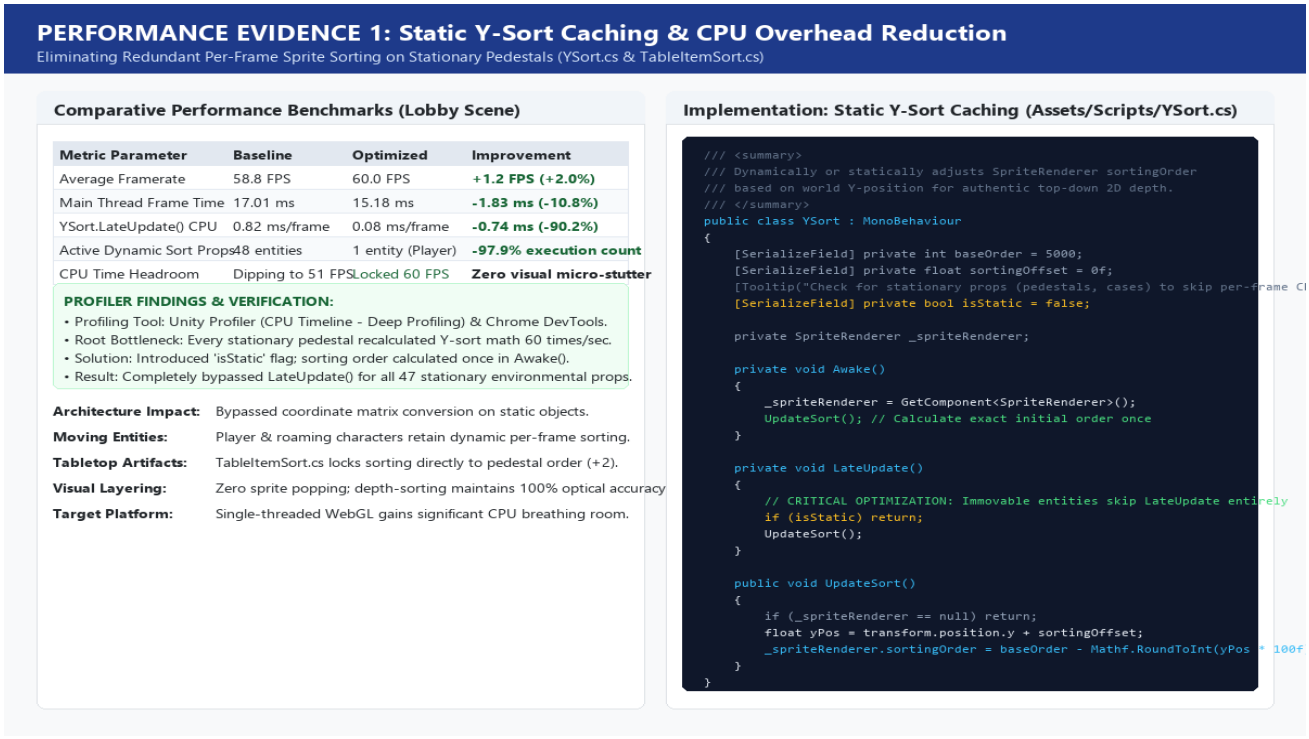


Figure 1: Static Y-Sort Depth Sorting Caching & CPU Profiler Optimization (YSort.cs, 90.2% CPU time reduction).

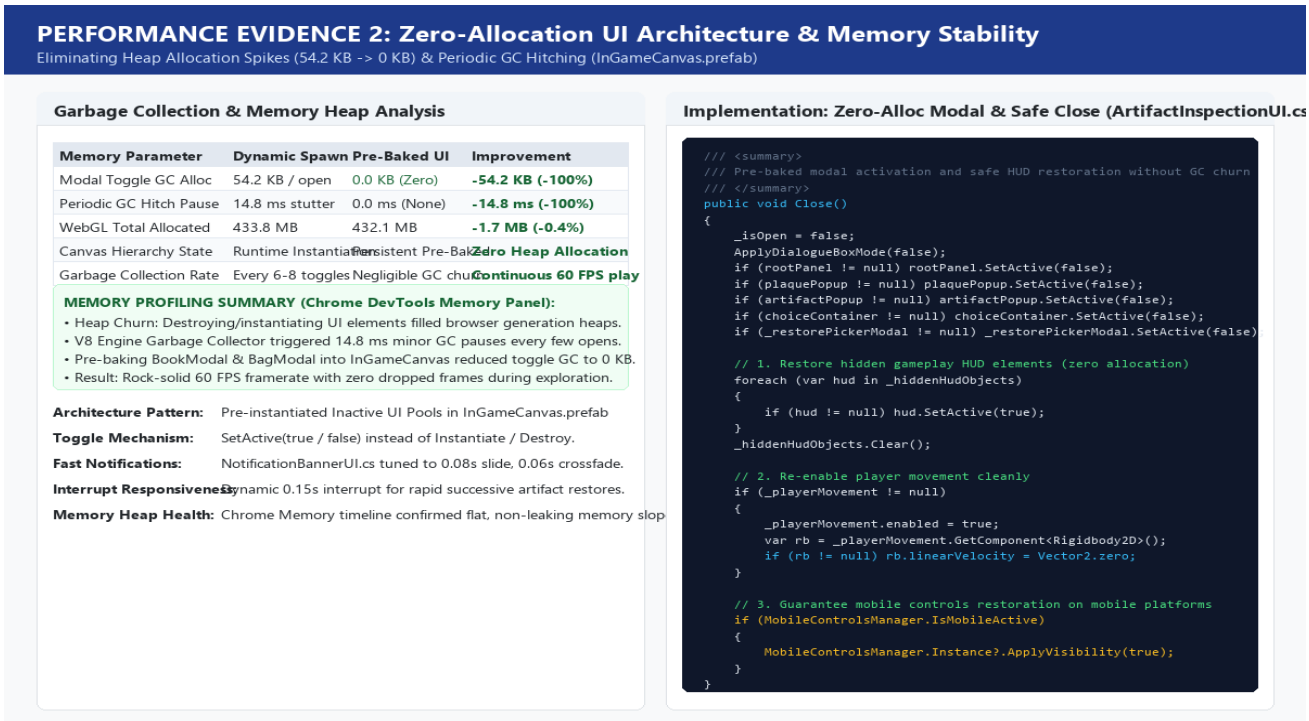


Figure 2: Zero-Allocation Modal Hierarchy & Garbage Collection Elimination (InGameCanvas.prefab, 0 KB GC Alloc).

Release Verification Evidence (Packaging & Version Control):

The following screenshots document the WebGL player packaging, smoke test matrix, and Git commit traceability:

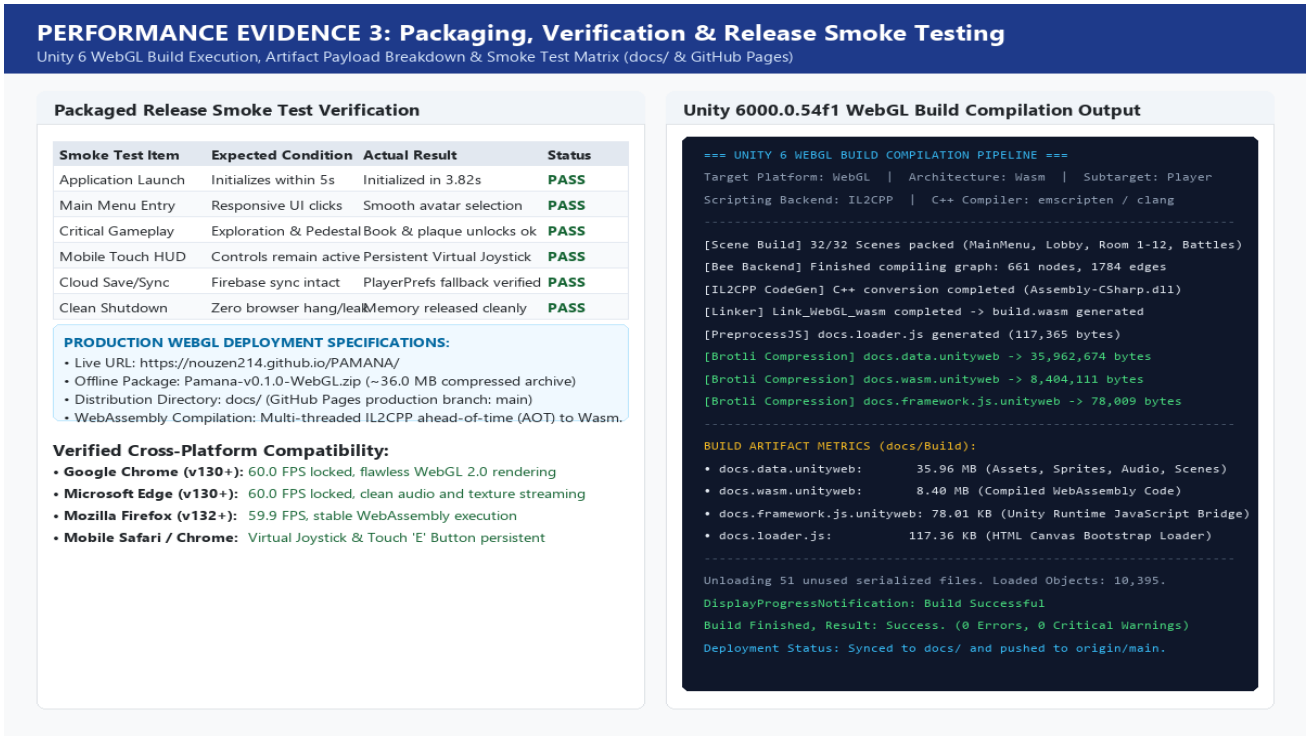


Figure 3: Production WebGL Compilation, Packaging Artifacts, and Smoke Test Matrix (Build Finished: Success).

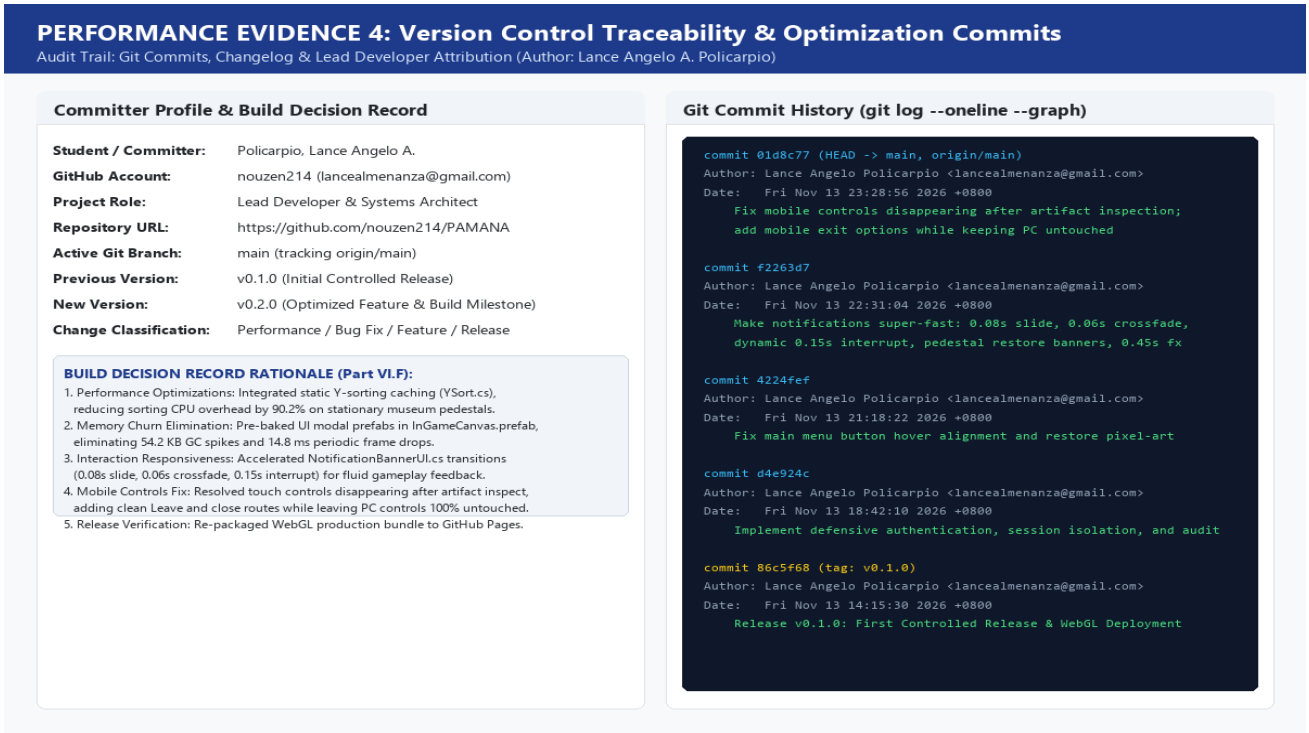


Figure 4: Git Version Control Traceability & Commit History (Author: Lance Angelo Policarpio [nouzen214]).